一、选择题

1.线性表采用链式存储结构时，其地址（　　）。

A. 必须是连续的　　B. 一定是不连续的

C. 部分地址必须是连续的　　D. 连续与否均可以

2. 线性表中最常用的操作是在最后一个数据元素之后插入一个数据元素和删除第一个数据元素，则采用（　　）存储方式最节省运算时间。

A. 单链表　　B. 仅有头指针的单循环链表

C. 双链表　　D. 仅有尾指针的单循环链表

3. 链表中结点的结构为（data,next）。已知指针 q 所指结点是指针 p 所指结点的直接前驱，若在 q 和 p 之间插入结点 s，则应执行下列（　）操作。

A. s->next=p->next; p->next=s;　　B. q->next=s; s->next=p;

C. p->next=s->next;s->next=p;　　D. p->next=s; s->next=q ;

4.链表不具有的特点是（　　）。

A.插入删除元素不需要移动元素　　B. 不必事先估计存储空间

C.可随机访问任何元素　　D.所需空间与线性表长度成正比

5.在下列顺序表进行的操作中，算法的时间复杂度为$O(1)$的是（　　）。

A.访问第i个元素的前驱（$1 < \mathrm{i} \leq \mathrm{n}$)　　B.在第i个元素后插入新元素（$1 \leq i \leq n$)

C.删除第i个元素（$1 \leq \mathrm{i} \leq \mathrm{n}$)　　D.对顺序表中元素进行排序

6.在头结点为head且表长大于 1 的单循环链表中，指针p指向表中某个结点，若p->next->next==head，则（　　）。

A. p 指向头结点　　B. p 指向尾结点

C. p 的直接后继是头结点　　D. p 的直接后继是尾结点

7.如果最常用的操作是取第i个结点及其前驱，则采用（　　）存储方式最节省时间。

A.单链表　　B.单循环链表　　C.双链表　　D.顺序表

8. 设单循环链表中结点的结构为（data,next），且 rear 是指向非空的带表头结点的单循环链表的尾结点的指针。若要删除链表的第一个结点，则应执行下列（　　）操作。

A. s=rear; rear=rear->next; delete s;

B. rear=rear->next; delete rear;

C. rear=rear->next->next; delete rear;

D. s=rear->next->next; rear->next->next=s->next; delete s;

9. 设双向循环链表中结点的结构为（pre,data,next）,且不带头结点。若在指针 p 所指结点之后插入结点 s，则应执行下列（　　）操作。

A. p->next=s; s->pre=p; p->next->pre=s; s->next=p->next;

B. p->next=s; p->next->pre=s; s->pre=p; s->next=p->next;

C. s->pre=p; s->next=p->next; p->next=s; p->next->pre=s;

D. s->pre=p; s->next =p->next; p->next->pre=s; p->next=s;

10.与单链表相比，双向链表的优点之一是（　　）。

A. 可以进行随机访问　　B. 插入、删除操作更简单

C. 访问前后相邻结点更灵活　　D. 可以省略表头指针或表尾指针

11.两个有序表分别具有 n 个元素与 m 个元素，且$\mathrm{n} \leq \mathrm{m}$，现将其合并成一个有序表，其最少的比较次数是（　　）。

NUST-CSE-2024-03

A.n　　B.m　　C.n-1　　D.m+n

12.在一个双向链表中，若 pr 是 p 的直接前驱结点，则删除 p 结点的正确的语句序列为（　　）。

A.pr->next=p->next;p->next->prior=pr;delete p;

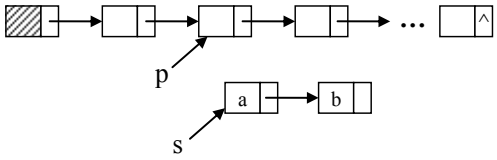
B.pr->next=p->next;if(p->next) p->next->prior=pr;delete p;

C.p->next->prior=pr; pr->next=p->next;delete p;

D. pr->next=p->next;delete p;

二、填空题

1.在如下图所示的链表中，若在指针p所指的结点之后插入数据域值相继为 a 和 b 的两个结点，请在后面的空格处填入正确的语句以实现该操作。_____; p->next = s;



2. 若线性表的数据元素总数基本稳定，很少进行插入和删除，但要求以最快的速度存取表中的数据元素，这时应采用_____存储表示。若经常进行插入删除操作，则该线性表应采用_____存储表示。

3.已知 L 是无表头结点的单链表，且 P 结点既不是首元结点，也不是尾结点，试从下列提供答案中选择合适的语句序列：

a）在 P 结点后插入 S 结点的语句序列是_____；

b）在 P 结点前插入 S 结点的语句序列是_____；

c）在表首插入 S 结点的语句序列是_____；

d）在表尾插入 S 结点的语句序列是_____；

(1)P->next=S;

(2)P->next=P->next->next;

(3)P->next=S->next;

(4)S->next=P->next;

(5)S->next=L;

(6)S->next=NULL;

(7)Q=P;

(8)while(P->next!=Q)P=P->next;

(9) while(P->next!=NULL)P=P->next;

(10)P=Q;

(11)P=L;

(12)L=S;

(13)L=P;

4.已知p结点是某双向链表的中间结点，试从下列提供的语句中选择合适的语句序列。

a）在p结点后插入s结点的语句序列是_____；

b）在p结点前插入s结点的语句序列是_____；

c）删除p结点的直接后继结点的语句序列是_____；

d）删除p结点的直接前驱结点的语句序列是_____；

e）删除p结点的语句序列是_____；

（1）p->next=p->next->next;

NUST-CSE-2024-03

（2）p->prior=p->prior->prior;
（3）p->next=s;
（4）p->prior=s;
（5）s->next=p;
（6）s->prior=p;
（7）s->next=p->next;
（8）s->prior=p->prior;
（9）p->prior->next=p->next;
（10）p->prior->next=p；
（11）p->next->prior=p;
（12）p->next->prior=s;
（13）p->prior->next=s;
（14）p->next->prior=p->prior;
（15）q=p->next;
（16）q=p->prior;
（17）delete p;
（18）delete q;

三、算法设计

1. 已知一个带头结点的单链表 ha 中存放一组整型数，构造一算法将链表 ha 中值为偶数的结点加入链表 hb 中，且链表 hb 按值非递减排列。函数原型为 void Inserthb(LNode *ha,LNode *&hb)。

2. 已知一个不带头结点的单链表 head 中存放一组整型数，构造一算法，删除链表中值最大的结点。

NJUST-CSE-2024-03

1. 已知一个带头结点的单链表 ha 中存放一组整型数，构造一算法将链表 ha 中值为偶数的结点加入链表 hb 中，且链表 hb 按值非递减排列。函数原型为void Inserthb(LNode *ha,LNode *&hb)。

```
void Inserthb(LNode *ha, LNode *&hb)
{   LNode *pa = ha->next;
    LNode *pb, *pre, *temp;
    while (pa)
    {   if (pa->data %2==0)
        {   temp = pa
            temp->data = pa->data;

            pre = hb;
            pb = hb->next;
            while (pb && pb->data < temp->data)
            {  pre = pb;
               pb = pb->next;
            }
            temp->next = pb;
            pre->next = temp;
        }
        pa = pa->next;
    }
}
```

hb = new LNode;
hb->next = NULL;

2. 已知一个不带头结点的单链表 head 中存放一组整型数，构造一算法，删除链表中值最大的结点。

```
void deleteMax(LinkList *head)
{  if (!head) return;
   if (head && head->next==NULL)
   {   delete head;
   }
   LNode *p = head, *pre = NULL, *max = head, *premax = NULL;
   while (p)
   {   if (p->data > max->data)
       {  max = p;
          premax = pre;
```

```
        }
        pre = p;
        p = p->next;
    }
    if( premax == NULL)
    {   head = max->next;
        delete max;
    }
    else
    {   premax->next = max->next;
        delete max;
    }
}
```